# AuthEngine — Comprehensive Notes

📋 What is this?

These notes cover the complete design and implementation of **AuthEngine** — a production-ready authentication service built with FastAPI, MongoDB (Beanie ODM), and JWT.

---

## Table of Contents

1. Architecture Philosophy
2. Tech Stack — Why Each Choice
3. Project Structure
4. Data Models
5. Pydantic Schemas
6. Configuration Management
7. Password Hashing — SHA-256 + bcrypt
8. JWT — Access & Refresh Tokens
9. Authentication Flows
10. Role-Based Access Control (RBAC)
11. FastAPI Dependency Injection
12. Email & OTP Utilities
13. Security Design Decisions
14. API Endpoint Reference
15. Environment Variables

---

## 1. Architecture Philosophy

AuthEngine follows **clean layered architecture**. The core rule is: *each layer knows only about the layer below it, never above*.

```
HTTP Request
     │
     ▼
┌──────────────┐
│   Routers    │  ← Receives HTTP, validates input shape, delegates
└──────┬───────┘
       │
       ▼
┌──────────────┐
│  Services    │  ← All business logic lives here
└──────┬───────┘
       │
       ▼
┌──────────────┐
│   Models     │  ← Database documents (Beanie ODM)
└──────────────┘
```

**Why this matters:**

- Routers never touch the database directly — they only call service functions

- Services never know about HTTP concepts like `Request`, `Response`, or status codes (except via `HTTPException` for error signalling)
- Models are pure data structures — no business logic embedded in them
- This separation makes each layer independently testable and replaceable

**Mental Model**

Think of it like a restaurant. The **waiter (router)** takes your order and passes it to the **chef (service)** who uses **ingredients from the pantry (models/DB)**. The waiter never opens the fridge themselves.

---

## 2. Tech Stack — Why Each Choice

| Layer | Technology | Why |
|---|---|---|
| **Web Framework** | FastAPI | Async-first, automatic OpenAPI docs, native Pydantic integration, dependency injection built-in |
| **Database** | MongoDB + Motor | Schema-flexible document store; Motor gives async I/O without blocking the event loop |
| **ODM** | Beanie | Async ODM on top of Motor; gives Pydantic models for MongoDB docs, supports validators, indexes, and migrations |
| **Auth** | `python-jose` | Industry-standard JWT encode/decode with full RFC 7519 compliance |
| **Hashing** | `passlib[bcrypt]` | bcrypt is the gold standard for password hashing — memory-hard, salted, slow by design |
| **Email** | `smtplib` (stdlib) | No extra dependency for basic SMTP; works with Gmail App Passwords |
| **Config** | `pydantic-settings` | Type-validated settings from `.env` files; fail-fast on startup if required vars are missing |

### Why FastAPI over Flask/Django?

- **Async native** — FastAPI is built on Starlette/ASGI. Every DB call in this project uses `await`, meaning one worker thread can handle thousands of concurrent requests
- **Pydantic integration** — Request bodies are automatically parsed and validated into Python objects. No `request.json()` parsing boilerplate
- **Auto-docs** — `/docs` (Swagger UI) and `/redoc` are generated automatically from your code
- **Dependency injection** — FastAPI's `Depends()` system is composable and avoids repetitive middleware code

### Why MongoDB over PostgreSQL?

- Auth schemas evolve frequently (adding fields, new token types) — MongoDB's flexible schema suits this
- No foreign key joins needed for auth data — documents are self-contained
- Motor's async driver is mature and production-tested

---

## 3. Project Structure

```
auth-system/
├── app/
│   ├── main.py              # FastAPI app creation, DB init, router registration
│   ├── routers/
│   │   └── auth.py          # HTTP endpoint definitions only
│   ├── services/
│   │   └── auth_service.py  # Business logic for all auth flows
│   ├── models/
│   │   ├── user.py          # User MongoDB document
│   │   └── token.py         # Token + RefreshToken MongoDB documents
│   ├── schemas/
│   │   └── auth.py          # Pydantic request/response schemas
│   ├── core/
│   │   ├── config.py        # Settings (env vars)
│   │   ├── security.py      # Password hashing
│   │   ├── jwt.py           # Token creation and decoding
│   │   └── dependencies.py  # FastAPI dependency functions
│   └── utils/
│       ├── email.py         # SMTP email sender
│       └── otp.py           # OTP/token generation helpers
├── .env.example             # Template for environment variables
├── .gitignore
└── requirements.txt
```

**`__init__.py` Files**

Every directory under `app/` has an empty `__init__.py`. This tells Python to treat each directory as a **package**, enabling absolute imports like `from app.core.config import settings`. Without these files, Python would not find the modules.

---

## 4. Data Models

Models are defined using **Beanie** which wraps Pydantic `BaseModel` with MongoDB persistence.

### 4.1 User Document

PYTHON

```python
class User(Document):
    email: EmailStr          # validated email format
    first_name: str
    last_name: str
    password_hash: str       # NEVER the plaintext password
    role: UserRole           # USER or ADMIN (enum)
    is_active: bool = False  # starts False until email verified
    is_verified: bool = False
    is_2fa_enabled: bool = False
    created_at: datetime

    class Settings:
        name = "users"       # MongoDB collection name
        indexes = ["email"]  # single-field index for fast lookup
```

**Key design decisions:**

| Decision | Reason |
|---|---|
| `is_active = False` on creation | A user cannot log in until they verify their email. Prevents fake accounts from being active immediately. |
| `is_verified` separate from `is_active` | In the future, an admin could deactivate an already-verified user without losing their verified status. They serve different purposes. |
| `password_hash` not `password` | The field name enforces the convention that we never store raw passwords — the name is a constant reminder. |
| `role` as an Enum | Prevents magic strings. `UserRole.ADMIN` is safer than `"admin"` — typos cause AttributeError at dev time, not a silent auth bypass in production. |

## 4.2 Token Document

```python
class TokenType(str, Enum):
    EMAIL_VERIFICATION = "email_verification"
    PASSWORD_RESET = "password_reset"
    TWO_FA_OTP = "2fa_otp"

class Token(Document):
    user_id: PydanticObjectId  # foreign key reference
    token: str                  # the actual token/OTP string
    type: TokenType             # what purpose this token serves
    expires_at: datetime        # hard expiry timestamp
    is_used: bool = False       # single-use enforcement
    created_at: datetime
```

**Why a single `Token` collection for three purposes?**
All three (email verification, password reset, 2FA OTP) share identical fields and lifecycle. Having one collection with a `type` discriminator is simpler than three separate collections with identical schemas.

**`is_used` flag — single-use enforcement:**
When a token is consumed, it is flagged `is_used = True` rather than deleted. This provides an audit trail: you can see that a token *was* used, when it was created, and by which user — useful for debugging and security investigations.

## 4.3 RefreshToken Document

```python
class RefreshToken(Document):
    user_id: PydanticObjectId
    token: str
    expires_at: datetime
    is_revoked: bool = False
    created_at: datetime
```

**Why store refresh tokens in the database at all?**

JWT refresh tokens are cryptographically verifiable on their own — you could skip the DB entirely. But storing them gives you:

1. **Revocation** — you can invalidate a token immediately (on logout, password reset, suspicious activity) without waiting for it to expire
2. **Token rotation tracking** — when you rotate a token, you mark the old one revoked and insert the new one; this creates an audit trail
3. **Detecting theft** — if an already-revoked refresh token is submitted, that's a signal of token theft (refresh token reuse detection)

> ⚠ **Security tradeoff**
>
> Pure stateless JWTs cannot be revoked. If a refresh token is stolen and you don't store them server-side, you cannot invalidate it until it expires — which could be 7–30 days. Storing them in DB is the correct production choice.

---

## 5. Pydantic Schemas

Schemas live in `app/schemas/auth.py` and are **separate** from the Beanie models. This is a critical distinction.

### Why separate schemas from models?

| Beanie Model (`app/models/`) | Pydantic Schema (`app/schemas/`) |
|---|---|
| Maps 1:1 to a MongoDB document | Defines the shape of HTTP request/response bodies |
| Contains `password_hash` | Never exposes `password_hash` to clients |
| Has MongoDB metadata (`Settings`) | Has input validation rules |
| Used by services and the DB layer | Used by routers only |

**Example — what the client sends vs. what gets stored:**

```
Client sends:        RegisterRequest
                     { email, password, first_name, last_name, role }
                              |
                              ▼ Service hashes the password
                     User (stored in DB)
                     { email, password_hash, first_name, last_name, role, is_active=False, ... }
```

The client **never** sees `password_hash`. The `UserResponse` schema returns only safe fields.

### Field Validators

```python
@field_validator("password")
@classmethod
def password_strength(cls, v: str) -> str:
    if len(v) < 8:
        raise ValueError("Password must be at least 8 characters")
    return v
```

Pydantic validators run at parse time, before the request even reaches your service function. FastAPI automatically returns a `422 Unprocessable Entity` with a detailed error message if validation fails. You write zero error-handling code in your router for this.

---

## 6. Configuration Management

```python
# app/core/config.py
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    MONGO_URI: str           # required — no default
    JWT_SECRET_KEY: str      # required — no default
    ACCESS_TOKEN_EXPIRE_MINUTES: int = 15   # optional with default
    ...
    class Config:
        env_file = ".env"

settings = Settings()        # instantiated once at import time
```

**How it works:**

1. `pydantic-settings` reads variables from the OS environment and from the `.env` file
2. Each field is **type-cast automatically** — `ACCESS_TOKEN_EXPIRE_MINUTES=15` becomes Python `int`, not string `"15"`
3. Required fields (no default) will raise a `ValidationError` at **startup** if missing — the app refuses to start with missing config. This is called **fail-fast** and is the correct production pattern.
4. The `settings` singleton is imported wherever needed: `from app.core.config import settings`

> ⚡ Never Hardcode Secrets
>
> `JWT_SECRET_KEY` is never set in the code. It only comes from the environment. If it were hardcoded and the repo became public, every token signed with that key would be compromised. `.env` files are in `.gitignore` precisely for this reason.

---

## 7. Password Hashing — SHA-256 + bcrypt

This is one of the most important security components. The approach uses a **two-step hashing pipeline**.

### The Problem with Plain bcrypt

bcrypt has a **72-byte input limit**. Any password longer than 72 bytes is silently truncated — meaning `password_that_is_73_chars_long` and `password_that_is_73_chars_lon` hash to the same value. Users with long passwords get a false sense of security.

### The Solution: SHA-256 Pre-hashing

```python
import hashlib
from passlib.context import CryptContext

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def hash_password(password: str) -> str:
    password_bytes = password.encode("utf-8")
    sha256_hash = hashlib.sha256(password_bytes).hexdigest()  # always 64 hex chars
    return pwd_context.hash(sha256_hash)                      # bcrypt that

def verify_password(password: str, hashed: str) -> bool:
    password_bytes = password.encode("utf-8")
    sha256_hash = hashlib.sha256(password_bytes).hexdigest()
    return pwd_context.verify(sha256_hash, hashed)
```

**The pipeline:**

```
User's password (any length)
         |
         ▼  hashlib.sha256(...).hexdigest()
SHA-256 hash (always exactly 64 hex characters = 64 bytes)
         |
         ▼  passlib bcrypt
bcrypt hash (stored in DB)
```

SHA-256 output is always 64 bytes — well within bcrypt's 72-byte limit. bcrypt then adds its own random **salt** and applies its slow key-derivation function.

### Why bcrypt specifically?

| Property | bcrypt | MD5/SHA | Plain SHA-256 |
|---|---|---|---|
| Salted | Yes (built-in) | Must add manually | Must add manually |
| Slow by design | Yes (work factor) | No | No |
| Rainbow table resistant | Yes | With salt | With salt |
| GPU-resistant | Yes (memory-hard) | No | No |

bcrypt is intentionally slow — it takes ~100ms to hash a password. This is a feature: an attacker who steals the database and tries to brute-force passwords can only attempt ~10 guesses/second per CPU core instead of millions.

> 🖊 `CryptContext` and `deprecated="auto"`
>
> `passlib.CryptContext` is a wrapper that handles algorithm upgrades transparently. The `deprecated="auto"` setting means: if you ever switch from bcrypt to argon2, old bcrypt hashes are automatically identified and can be re-hashed on next login — without forcing a mass password reset.

---

## 8. JWT — Access & Refresh Tokens

### What is a JWT?

A **JSON Web Token** is a Base64URL-encoded string with three dot-separated parts:

```
eyJhbGciOiJIUzI1NiJ9  .  eyJzdWIiOiI2NjQ...  .  SflKxwRJSMeKKF2QT4fwpM
      Header                    Payload                Signature
```

- **Header**: algorithm (`HS256`)
- **Payload**: claims — `sub` (user ID), `role`, `exp` (expiry), `type`
- **Signature**: HMAC-SHA256 of `header.payload` using the secret key

The server can verify any token without a database lookup — just re-compute the signature and compare. If it matches and `exp` hasn't passed, the token is valid.

### Access Token

```python
def create_access_token(user_id: str, role: str) -> str:
    expire = datetime.now(timezone.utc) + timedelta(minutes=15)
    payload = {
        "sub": user_id,     # "subject" — standard JWT claim
        "role": role,
        "exp": expire,      # standard JWT expiry claim
        "type": "access"    # custom claim — prevents token type confusion
    }
    return jwt.encode(payload, settings.JWT_SECRET_KEY, algorithm="HS256")
```

**Short-lived (15 minutes)** — minimises the damage window if an access token is stolen. An attacker who intercepts it can only use it for 15 minutes.

### Refresh Token

```python
def create_refresh_token(user_id: str) -> str:
    expire = datetime.now(timezone.utc) + timedelta(days=7)
    payload = {"sub": user_id, "exp": expire, "type": "refresh"}
    return jwt.encode(payload, settings.JWT_SECRET_KEY, algorithm="HS256")
```

**Long-lived (7 days)** — used only to get a new access token. Stored in the database so it can be revoked.

### The `type` Claim — Token Confusion Prevention

```python
# In decode_access_token:
if payload.get("type") != "access":
    return None
```

This check prevents **token type confusion attacks**. Without it, an attacker could submit a refresh token where an access token is expected (or vice versa). The `type` field makes each token only valid for its intended purpose.

## Token Lifecycle Diagram

```
┌──────────────────────────────────────────────────────────┐
│                        Login                             │
│  Client ──POST /auth/login──▶ Server                     │
│         ◀── access_token (15min) + refresh_token (7d) ── │
└──────────────────────────────────────────────────────────┘
                          │
                    [15 min later]
                          │
                          ▼
┌──────────────────────────────────────────────────────────┐
│                    Token Refresh                         │
│  Client ──POST /auth/refresh (refresh_token)──▶ Server   │
│         ◀── NEW access_token + NEW refresh_token ──────  │
│         (old refresh token is REVOKED in DB)             │
└──────────────────────────────────────────────────────────┘
                          │
                    [User logs out]
                          │
                          ▼
┌──────────────────────────────────────────────────────────┐
│                        Logout                            │
│  Client ──POST /auth/logout (refresh_token)──▶ Server    │
│         (refresh token marked is_revoked=True in DB)     │
└──────────────────────────────────────────────────────────┘
```

## Refresh Token Rotation

On every `/auth/refresh` call:

1. Old refresh token is marked `is_revoked = True`
2. A brand new refresh token is issued and stored

**Why rotate?** If a refresh token is stolen, the attacker uses it once — but so does the legitimate user. The moment the legitimate user refreshes again, the stolen token is revoked. The second use of any given refresh token is a signal of theft.

---

# 9. Authentication Flows

## 9.1 Registration + Email Verification

```
Client                          Server                        DB
  |                               |                            |
  |──POST /auth/register ────────►|                            |
  |                               |── Check email exists ─────►|
  |                               |◄── (none found) ───────────|
  |                               |── Insert User(is_active=F)►|
  |                               |── Insert Token(EMAIL_VERIF)►|
  |                               |── send_verification_email() |
  |◄── 201 { message } ──────────| |                            |
  |                               |                            |
  |  [User clicks email link]     |                            |
  |                               |                            |
  |──GET /auth/verify-email?token►|                            |
  |                               |── Find Token (unused) ────►|
  |                               |── Update User.is_active=T ►|
  |                               |── Update Token.is_used=T ──►|
  |◄── 200 { message } ──────────| |                            |
```

**Key security details:**

- New users start with `is_active=False` and `is_verified=False`
- The verification token is a `secrets.token_urlsafe(32)` — 32 bytes = 256 bits of entropy, effectively unguessable
- Token expires after 24 hours — prevents old links from working indefinitely
- Token is single-use — once verified, `is_used=True` prevents replay

## 9.2 Login (with 2FA branch)

```
Client                    Server
  |                         |
  |──POST /auth/login ──────►|
  |                         |  1. Find user by email
  |                         |  2. verify_password(input, stored_hash)
  |                         |  3. Check is_verified + is_active
  |                         |
  |                         ├── [2FA disabled] ──────────────────────►
  |                         |   Create access + refresh tokens
  |◄── TokenResponse ───────┤   Store refresh token in DB
  |                         |
  |                         └── [2FA enabled] ───────────────────────►
  |                             Generate 6-digit OTP
  |                             Store OTP in Token collection
  |                             send_2fa_otp_email()
  |◄── { message: "OTP sent" }─|
  |                         |
  |──POST /auth/verify-2fa──►|  Validate OTP (unused, not expired)
  |                         |  Mark OTP as used
  |◄── TokenResponse ───────|  Issue access + refresh tokens
```

**Why validate `is_verified` and `is_active` after password check, not before?**

You could check these first (before even looking up the password), but that would leak information: an attacker could determine "this email is registered and verified" vs "this email doesn't exist". By always checking the password first and

returning the same `"Invalid credentials"` error for all failure modes, you prevent **user enumeration**.

> ⚠ **Credential Error Messages**
>
> Never return "Wrong password" vs "Email not found" — always return "Invalid credentials" for any credential failure. The message is the same whether the email doesn't exist, the password is wrong, or the account is inactive at the credential check stage.

## 9.3 Forgot Password / Reset Password

```
Client                    Server
  |                         |
  |──POST /forgot-password──→|  Find user by email
  |                         |  [If user not found: still return same message]
  |                         |  Generate 6-digit OTP
  |                         |  Store in Token collection (PASSWORD_RESET type)
  |                         |  Send OTP via email
  |←── { "If that email exists..." }
  |                         |
  |──POST /reset-password ──→|  { token: "123456", new_password: "..." }
  |                         |  Find token (is_used=False, type=PASSWORD_RESET)
  |                         |  Check not expired
  |                         |  hash_password(new_password)
  |                         |  Save new password_hash to user
  |                         |  Mark OTP as used
  |                         |  Revoke ALL active refresh tokens for user
  |←── { message } ─────────|
```

**Anti-enumeration in forgot-password:**

```python
if not user:
    return {"message": "If that email exists, an OTP has been sent."}
```

The response is identical whether or not the email exists. An attacker submitting thousands of emails cannot determine which ones are registered in your system.

**Revoking all refresh tokens after password reset:**

```python
async for rt in RefreshToken.find(RefreshToken.user_id == user.id, ...):
    rt.is_revoked = True
    await rt.save()
```

If someone has compromised your account and triggered a password reset, you don't want their existing sessions to remain valid. Revoking all tokens forces every device to re-authenticate with the new password.

## 9.4 Refresh Token Flow (Rotation)

```python
async def refresh_tokens(data: RefreshRequest) -> TokenResponse:
    user_id = decode_refresh_token(data.refresh_token)  # verify JWT signature
    if not user_id:
        raise HTTPException(401, "Invalid refresh token")

    stored = await RefreshToken.find_one(
        RefreshToken.token == data.refresh_token,
        RefreshToken.is_revoked == False,
    )
    # Double validation: JWT cryptography AND database presence
    if not stored or stored.expires_at < now:
        raise HTTPException(401, "Expired or revoked")

    stored.is_revoked = True   # revoke old token
    await stored.save()

    # Issue new pair
    new_access = create_access_token(...)
    new_refresh = create_refresh_token(...)
    await RefreshToken(token=new_refresh, ...).insert()

    return TokenResponse(access_token=new_access, refresh_token=new_refresh)
```

**Double validation explained:**

1. `decode_refresh_token()` — verifies the JWT signature and expiry cryptographically
2. `RefreshToken.find_one(is_revoked=False)` — checks the database to confirm it hasn't been revoked

You need both checks. The JWT check alone doesn't catch revoked tokens. The DB check alone doesn't verify the signature. Together they provide full validation.

## 9.5 Two-Factor Authentication (2FA)

**Enabling 2FA** requires the user to be authenticated (uses `get_current_user` dependency):

```python
@router.post("/enable-2fa")
async def enable_2fa(
    data: Enable2FARequest,          # { enable: true/false }
    current_user: User = Depends(get_current_user),  # must be logged in
):
```

**2FA OTP Flow:**

- OTP is a 6-digit numeric string from `secrets.choice(string.digits)`
- `secrets` module (not `random`) — cryptographically secure random
- Expires in 10 minutes (short window to prevent brute-force)
- Single-use (`is_used=True` after successful verify)
- Stored in the same `Token` collection with type `TWO_FA_OTP`

---

## 10. Role-Based Access Control (RBAC)

### The Roles

```python
class UserRole(str, Enum):
    USER = "USER"
    ADMIN = "ADMIN"
```

`str, Enum` means `UserRole.USER == "USER"` evaluates to `True`. This makes the enum serializable as a plain string in JSON responses and comparable to string values from the database, while still being a proper enum in Python code.

### How the role is embedded in the JWT

```python
payload = {"sub": user_id, "role": role, "exp": expire, "type": "access"}
```

The role is encoded into the access token at login time. This means **role checks are stateless** — the server doesn't need to query the database to check a user's role on every request. The role is read directly from the verified token payload.

> **Role staleness**
>
> If a user's role changes in the DB while they have an active access token, the token still carries the old role until it expires (15 minutes). For most systems this is acceptable. If immediate revocation is needed, you'd need to check role against DB on every request.

---

## 11. FastAPI Dependency Injection

This is one of FastAPI's most powerful features. Dependencies are functions that FastAPI automatically calls and injects before your route handler runs.

### `get_current_user`

```python
bearer_scheme = HTTPBearer()  # expects "Authorization: Bearer <token>" header

async def get_current_user(
    credentials: HTTPAuthorizationCredentials = Depends(bearer_scheme),
) -> User:
    token = credentials.credentials
    payload = decode_access_token(token)
    if not payload:
        raise HTTPException(401, "Invalid or expired token")
    user = await User.get(payload["sub"])
    if not user or not user.is_active:
        raise HTTPException(401, "User not found or inactive")
    return user
```

Any route that declares `current_user: User = Depends(get_current_user)` automatically:

1. Requires the `Authorization: Bearer <token>` header
2. Validates the token
3. Fetches the user from MongoDB
4. Returns `401` if anything fails
5. Passes the `User` object to your handler if everything passes

## `require_roles` — Dependency Factory

```python
def require_roles(*roles: UserRole):
    async def checker(current_user: User = Depends(get_current_user)) -> User:
        if current_user.role not in roles:
            raise HTTPException(403, "Insufficient permissions")
        return current_user
    return checker
```

This is a **factory function** — it returns a dependency. Note that `checker` itself depends on `get_current_user`, so dependencies are composable and chainable.

**Usage:**

```python
# Route that only ADMINs can access
@router.get("/admin-only")
async def admin_only(current_user: User = Depends(require_roles(UserRole.ADMIN))):
    ...

# Route that ADMINs or MODERATORs could access (if you add MODERATOR role later)
async def mod_or_admin(user = Depends(require_roles(UserRole.ADMIN, UserRole.MODERATOR))):
    ...
```

**Why not use middleware for auth?**

Middleware applies to all routes. Dependency injection lets you apply auth to specific routes only, with different permission requirements per route. It's more granular and explicit.

## Dependency Chain Diagram

```
Request hits /auth/admin-only
         |
         ▼
require_roles(ADMIN) called
         |
         ▼ (depends on)
get_current_user called
         |
         ▼ (depends on)
HTTPBearer() extracts token from header
         |
         ▼
Token decoded, user fetched from DB
         |
         ▼
Role checked against [ADMIN]
         |
         ▼
Handler receives `current_user` object
```

---

# 12. Email & OTP Utilities

## OTP Generation

```python
import secrets
import string

def generate_otp() -> str:
    return "".join(secrets.choice(string.digits) for _ in range(6))

def generate_token() -> str:
    return secrets.token_urlsafe(32)
```

`secrets` vs `random`:

| | `random` | `secrets` |
|---|---|---|
| Purpose | General-purpose randomness | Cryptographically secure randomness |
| Predictable? | Yes (seeded PRNG) | No (OS entropy source) |
| Use in security? | Never | Always |

`secrets.token_urlsafe(32)` produces 32 bytes (256 bits) of random data, Base64URL-encoded. The probability of guessing it is `1/(2^256)` — computationally infeasible.

## Email Sending (SMTP)

```python
def _send(to: str, subject: str, html_body: str) -> None:
    with smtplib.SMTP(settings.SMTP_HOST, settings.SMTP_PORT) as server:
        server.starttls()    # upgrade to TLS
        server.login(settings.SMTP_USER, settings.SMTP_PASSWORD)
        server.sendmail(settings.EMAIL_FROM, to, msg.as_string())
```

`starttls()` — starts the connection on plain port 587 and upgrades to TLS immediately. This is the modern SMTP approach (vs. port 465 which uses TLS from the start).

**Gmail Setup**

Use a **Gmail App Password** (not your Google account password). Go to Google Account → Security → 2-Step Verification → App passwords. This generates a 16-character password that only works for SMTP.

**Synchronous Email Sending**

The current implementation sends email synchronously — the HTTP request waits for the email to be delivered before responding. In production, move this to a background task (FastAPI's `BackgroundTasks` or a Celery queue) so that a slow SMTP server doesn't make your API slow.

---

# 13. Security Design Decisions

## 13.1 Summary Table

| Threat | Defence |
|---|---|
| Password theft via DB breach | bcrypt + SHA-256 pre-hash — hashes are useless without cracking |

| Threat | Defence |
|---|---|
| Password brute-force on login | Rate limiting (via `slowapi`) |
| Stolen access token | Short expiry (15 min) |
| Stolen refresh token | DB storage + revocation + rotation |
| Email enumeration | Identical error messages for all credential failures |
| OTP brute-force | 6-digit OTP with 10-minute expiry + single-use |
| Token type confusion | `type` claim in JWT payload |
| Secrets in source code | All secrets from env vars, never hardcoded |
| Long password bcrypt truncation | SHA-256 pre-hash normalises input to 64 bytes |
| Session persistence after password change | Revoke all refresh tokens on password reset |
| Replay attacks with verification tokens | `is_used = True` flag, single-use enforcement |

## 13.2 What `is_used` Prevents

Without `is_used`:

1. User clicks email verification link → account verified
2. Attacker captures the verification URL (from browser history, logs, etc.)
3. Attacker can re-submit the token later — no effect since account is already verified, but it would be a vulnerability if the token could be used to trigger a privileged action

With `is_used = True` after first use, any second submission returns `"Invalid or expired token"`.

## 13.3 The `timezone.utc` Pattern

```python
# BAD — naive datetime (no timezone info)
datetime.utcnow()

# GOOD — timezone-aware datetime
datetime.now(timezone.utc)
```

`datetime.utcnow()` returns a naive datetime (no timezone info). When you compare it to a timezone-aware datetime from the database, Python raises a `TypeError`. The project uses `datetime.now(timezone.utc)` throughout to ensure all comparisons are between timezone-aware objects.

This is also why the token expiry check does:

```python
token.expires_at.replace(tzinfo=timezone.utc) < datetime.now(timezone.utc)
```

The `.replace(tzinfo=timezone.utc)` adds timezone info to a datetime that may have been stored and retrieved without it.

## 13.4 Async/Await Throughout

```python
# Every DB operation is awaited
user = await User.find_one(User.email == data.email)
await user.insert()
await user.save()
```

FastAPI runs on an async event loop. If you call a blocking operation (like a synchronous DB call) without `await`, you **block the entire event loop** — no other request can be processed until that operation completes. Using Motor (async MongoDB driver) + Beanie with `await` keeps the event loop free to handle concurrent requests.

---

## 14. API Endpoint Reference

| Method | Path | Auth Required | Description |
|---|---|---|---|
| `POST` | `/auth/register` | No | Register new user, sends verification email |
| `GET` | `/auth/verify-email?token=` | No | Activate account via email link |
| `POST` | `/auth/login` | No | Login; returns tokens or triggers 2FA |
| `POST` | `/auth/refresh` | No (refresh token in body) | Rotate refresh token, get new access token |
| `POST` | `/auth/logout` | No (refresh token in body) | Revoke refresh token |
| `POST` | `/auth/forgot-password` | No | Send OTP to email for password reset |
| `POST` | `/auth/reset-password` | No | Submit OTP + new password |
| `POST` | `/auth/enable-2fa` | Yes (Bearer) | Toggle 2FA on/off |
| `POST` | `/auth/verify-2fa` | No | Submit 2FA OTP, receive tokens |
| `GET` | `/auth/me` | Yes (Bearer) | Get current user's profile |
| `GET` | `/auth/admin-only` | Yes (Bearer, ADMIN role) | RBAC example — admin only |
| `GET` | `/health` | No | Service health check |

### Request/Response Examples

**Register:**

```
POST /auth/register
{
  "email": "user@example.com",
  "password": "securepass123",
  "first_name": "Kirk",
  "last_name": "Yagami",
  "role": "USER"
}

201 { "message": "Registration successful. Check your email to verify your account." }
```

**Login:**

```
POST /auth/login
{ "email": "user@example.com", "password": "securepass123" }

200 {
  "access_token": "eyJ...",
  "refresh_token": "eyJ...",
  "token_type": "bearer"
}
```

**Authenticated request:**

```
GET /auth/me
Authorization: Bearer eyJ...

200 {
  "id": "664abc...",
  "email": "user@example.com",
  "first_name": "Kirk",
  "last_name": "Yagami",
  "role": "USER",
  "is_active": true,
  "is_verified": true,
  "is_2fa_enabled": false
}
```

---

## 15. Environment Variables

| Variable | Required | Default | Description |
|---|---|---|---|
| `MONGO_URI` | Yes | — | MongoDB connection string |
| `DATABASE_NAME` | No | `auth_db` | MongoDB database name |
| `JWT_SECRET_KEY` | Yes | — | HS256 signing secret — must be long and random |
| `JWT_ALGORITHM` | No | `HS256` | JWT signing algorithm |
| `ACCESS_TOKEN_EXPIRE_MINUTES` | No | `15` | Access token lifetime |
| `REFRESH_TOKEN_EXPIRE_DAYS` | No | `7` | Refresh token lifetime |
| `SMTP_HOST` | No | `smtp.gmail.com` | SMTP server host |
| `SMTP_PORT` | No | `587` | SMTP port (587 = STARTTLS) |
| `SMTP_USER` | Yes | — | SMTP username (email address) |
| `SMTP_PASSWORD` | Yes | — | SMTP password or App Password |
| `EMAIL_FROM` | Yes | — | From address on sent emails |

**Generating a strong `JWT_SECRET_KEY`:**

```shell
python -c "import secrets; print(secrets.token_hex(32))"
# → 64 character hex string = 256 bits of entropy
```

---

# Appendix — Key Library Reference

## Beanie ODM Cheatsheet

```python
# Insert
await user.insert()

# Find one
user = await User.find_one(User.email == "x@y.com")

# Find by ID
user = await User.get("664abc...")

# Find many
async for rt in RefreshToken.find(RefreshToken.user_id == uid):
    ...

# Update
user.is_active = True
await user.save()
```

## JWT Claims (RFC 7519)

| Claim | Name | Meaning |
|---|---|---|
| `sub` | Subject | Who the token is about (user ID) |
| `exp` | Expiration Time | Unix timestamp after which token is invalid |
| `iat` | Issued At | When the token was created |
| `type` | (custom) | `"access"` or `"refresh"` — our own claim |
| `role` | (custom) | User's RBAC role — our own claim |

## HTTP Status Codes Used

| Code | Meaning | When used |
|---|---|---|
| `200` | OK | Successful GET/POST |
| `201` | Created | Registration |
| `400` | Bad Request | Invalid/expired token or OTP |
| `401` | Unauthorized | Invalid credentials, expired token |
| `403` | Forbidden | Email not verified, account inactive, insufficient role |
| `404` | Not Found | User not found |
| `409` | Conflict | Email already registered |
| `422` | Unprocessable Entity | Pydantic validation failure (automatic) |

---

*AuthEngine — built with FastAPI · MongoDB · JWT · bcrypt · python-jose · Beanie*